Secure Software Engineering – 20CYS401

Lab Exercise 1 — Refactoring Techniques for Enhancing Code Maintainability in Agile Projects

Praneesh R V

CB.SC.U4CYS23036

Task 1 - `auth_legacy.py` (user registration & login)

Techniques applied

#	Technique	Smell removed	Result
1	Replace Magic Number with Symbolic Constant	Hardcoded `4`, `20`, `3`, salt string	Named constants at top of file
2	Extract Variable (explaining variable)	Complex inline `any(c.isupper() ...)`	`has_upper`, `has_digit`
3	Replace Nested Conditionals with Guard Clauses	Deeply nested “arrow code” strength check	`check_password_strength()`
4	Extract Method	Duplicated username-length check (2 places)	`is_valid_username()`
5	Extract Method	Duplicated `hashlib.md5(...)` logic (2 places)	`generate_hash()`

1. Replace Magic Numbers with Symbolic Constants

Before

```
if len(username) < 4: ...
if len(username) > 20: ...
if user["failed_attempts"] >= 3: ...
salt = "s3cr3t_legacy_salt_2011"
```

After

```
MIN_USERNAME_LENGTH = 4
MAX_USERNAME_LENGTH = 20
MIN_PASSWORD_LENGTH = 8
MAX_FAILED_ATTEMPTS = 3
LEGACY_SALT = "s3cr3t_legacy_salt_2011"
```

2. Extract Variable to Explain Complex Logic

Store the result of a complex condition in a well-named boolean before using it — the name documents intent.

```
has_upper = any(c.isupper() for c in password)
has_digit = any(c.isdigit() for c in password)
```

3. Replace Nested Conditionals with Guard Clauses

Before — nested "arrow code"

```
strength = "WEAK"
if len(password) >= 8:
    if any(c.isupper() for c in password):
        if any(c.isdigit() for c in password):
            strength = "STRONG"
        else:
            strength = "MEDIUM"
```

After — early-return guard clauses

```
def check_password_strength(password):
    if len(password) < MIN_PASSWORD_LENGTH:
        return "WEAK"
    has_upper = any(c.isupper() for c in password)
    if not has_upper:
        return "WEAK"
    has_digit = any(c.isdigit() for c in password)
    if not has_digit:
        return "MEDIUM"
    return "STRONG"
```

4. Extract Method — duplicate username check

The exact same 4-line length check existed in both `register_user` and `login_user`.

```
def is_valid_username(username):
    return MIN_USERNAME_LENGTH <= len(username) <= MAX_USERNAME_LENGTH
```

5. Extract Method — duplicate hashing

Both functions repeated the same salted MD5 logic.

```python
def generate_hash(value):
    return hashlib.md5((value + LEGACY_SALT).encode()).hexdigest()
```

Task 1 verification

```
python3 -m pytest test_auth.py -q   ->   32 passed
```

32 cases (registration guards, strength tiers, ordering, hash contract, login lockout, failed-attempt counter) run against **both** `auth_legacy` and `auth_refactored` with identical results. Behavior preserved.

Task 2 - `filter-legacy.py` (mini-WAF security middleware)

Filters incoming JSON requests for SQLi / XSS / command-injection patterns, enforces a rate limit, validates payload integrity.

Smells found

#	Code Smell	Location	Refactoring Technique
1	Magic Numbers	`request_count > 5`, `len(val) > 100`	Replace Magic Number with Symbolic Constant
2	Scattered string literals	three injection `if` blocks	Introduce data table + `any()`
3	Long Method	whole `process_request`	Extract Method (`scan_value`)
4	Duplicated conditional structure	SQLi / XSS / CMD blocks	Consolidate into rule table
5	Dead (commented-out) code	`# temp_log_val = 999 ...`	Remove Dead Code
6	Unused import	`import time`	Remove Dead Code
7	Scope leak — loop var used after loop	`if len(val) > 100`	Flag (behavior-preserving)
8	Script side-effects at import	trailing `# Example usage`	Extract to `__main__` guard

Smells 1, 2 & 4 - Magic numbers + duplicated pattern checks

Before

```
if "SELECT" in val or "DROP" in val or "DELETE" in val or "UPDATE" in val or
"INSERT" in val:
    return "ERR_SQLI", 0
if "<script>" in val or "alert(" in val or "onerror" in val:
    return "ERR_XSS", 0
if ";" in val or "&&" in val or "||" in val or "|" in val or "rm -rf" in val:
    return "ERR_CMD_INJ", 0
```

Why a smell: three blocks share the same shape; patterns are buried in `or`-chains; `... in val` repeated 13 times. Adding a signature means editing a long boolean.

After - data-driven rule table (order preserved)

```
RATE_LIMIT = 5
MAX_VALUE_LENGTH = 100
SQLI_PATTERNS    = ["SELECT", "DROP", "DELETE", "UPDATE", "INSERT"]
XSS_PATTERNS     = ["<script>", "alert(", "onerror"]
CMD_INJ_PATTERNS = [";", "&&", "||", "|", "rm -rf"]
INJECTION_RULES  = [(SQLI_PATTERNS, "ERR_SQLI"),
                    (XSS_PATTERNS, "ERR_XSS"),
                    (CMD_INJ_PATTERNS, "ERR_CMD_INJ")]
```

Smell 3 - Long Method -> Extract Method

```
def scan_value(val):
    """Return the matching error code for a field value, or None if clean."""
    for patterns, error_code in INJECTION_RULES:
        if any(pattern in val for pattern in patterns):
            return error_code
    return None
```

Smell 5 & 6 - Dead code

`# temp_log_val = 999 # LEGACY LOGGING: DO NOT TOUCH` and `import time` are never used. *Technique:* Remove Dead Code — delete both. Version control preserves history.

Smell 7 - Scope leak (the interesting one)

```
for key in data:
    val = data[key]
    ...
if len(val) > 100:   # val = the LAST field's value only
    return "ERR_LEN", 0
```

Why a smell: `val` is a loop variable read after the loop, so the length check validates only the **last** field; an empty `data` dict raises `NameError`. A latent bug.

Golden-Rule handling: the **behavior-preserving** move is to keep it and add a comment making the “last value only” intent explicit. The **correct** fix (move the check inside the loop + guard empty dict) **changes external behavior**, so it is **flagged as a bug ticket, not applied** in this refactoring.

Smell 8 — Import-time side-effects → `__main__` guard

```python
if __name__ == "__main__":
    req_data = {"user": "admin", "query": "SELECT * FROM users"}
    status, count = process_request(req_data, 1)
    print(f"Result: {status}")
```

Why a smell: top-level demo code ran on import — printed to stdout and executed a request, making the module unusable as an importable library.

Task 2 verification

```
python3 -m pytest -q   ->   50 passed
```

`test_filter.py` runs the original `filter-legacy.py` and `filter_refactored.py` through the same inputs (rate limit, null, SQLi, XSS, command injection, clean, oversized-last-value, ordering, empty-dict `NameError`) — identical results. Structure changed, behavior did not.